
Foundations of Deep Learning

Lecture 02

Approximation And Complexity Theory

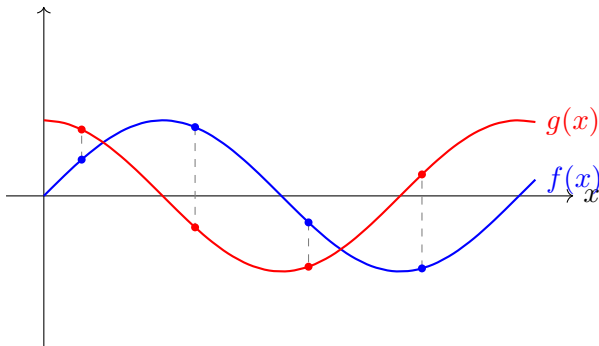
Aurelien Lucchi

Fall 2026

SECTION 1

Universality

How do we measure the quality of approximation?



How do we measure the quality of approximation?

ℓ_p norm

$$\|f\|_p = \begin{cases} \left(\int_S |f|^p \, d\mu\right)^{1/p} & 1 \leq p < \infty \\ \sup_{x \in S} |f(x)| & p = \infty \end{cases} < \infty.$$

$$\text{approx-err}(f, g) := \|f - g\|_p.$$

Extension to approximating function classes $\mathcal{G}$

$$\text{approx-err}(f, \mathcal{G}) := \inf \{g \in \mathcal{G} : \text{approx-err}(f, g)\} .$$

Consider family of models $\mathcal{G}_m$, $m = 1, 2, \dots$; $\mathcal{G} = \bigcup_m \mathcal{G}_m$.

E.g. MLPs with m hidden units.

$$f \simeq \mathcal{G} : \quad \text{approx-err}(f, \mathcal{G}) = 0$$

Clearly:

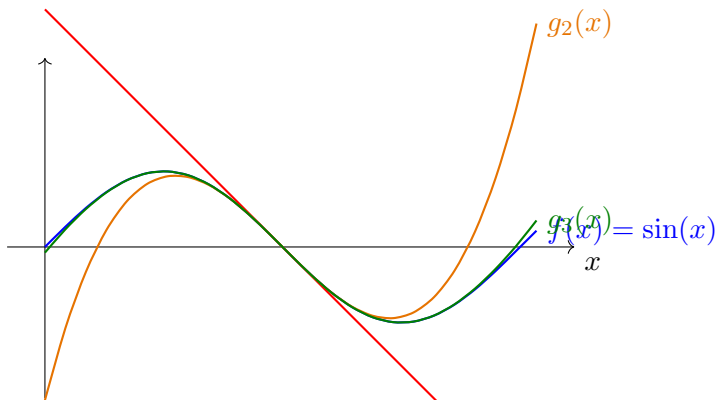
$$\Rightarrow \quad f \in \mathcal{G} \quad \Longrightarrow \quad f \simeq \mathcal{G}.$$

But also:

$$\Rightarrow \quad \mathcal{G} \ni g_m \xrightarrow{\text{unif.}} f \Longrightarrow f \simeq \mathcal{G}$$

where **uniform convergence** is defined as

$$(g_m) \xrightarrow{\text{unif.}} f \iff \forall \epsilon > 0 : \exists m \geq 1 : \|g_m - f\|_\infty < \epsilon.$$



$$g_1(x) = -(x - \pi),$$

$$g_2(x) = -(x - \pi) + \frac{(x - \pi)^3}{3!},$$

$$g_3(x) = -(x - \pi) + \frac{(x - \pi)^3}{3!} - \frac{(x - \pi)^5}{5!} + \frac{(x - \pi)^7}{7!}.$$

Generalizing to function classes: denseness

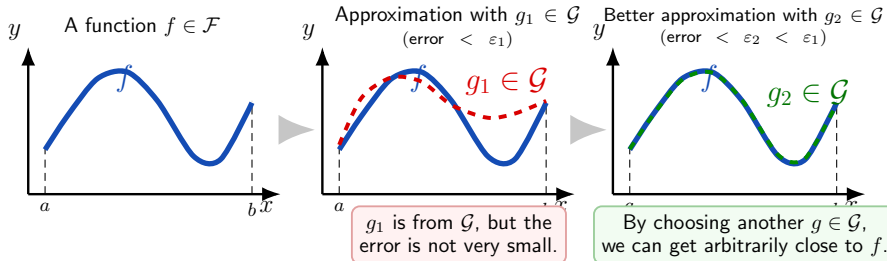
$$\begin{aligned} \mathcal{G} \subseteq \mathcal{F} \text{ is dense in } \mathcal{F} &\iff \mathcal{F} \simeq \mathcal{G} \\ &\iff \\ \forall f \in \mathcal{F} : f &\simeq \mathcal{G} \end{aligned}$$

The largest class of approximated functions is the closure $\text{cl}(\mathcal{G})$.

$$\Rightarrow \quad \text{cl}(\mathcal{G}) \simeq \mathcal{G} \quad \text{and} \quad \mathcal{F} \simeq \mathcal{G} \implies \mathcal{F} \subseteq \text{cl}(\mathcal{G})$$

Denseness of functions: $\mathcal{G}$ is dense in $\mathcal{F}$

For every function $f \in \mathcal{F}$ and every $\varepsilon > 0$, there exists $g \in \mathcal{G}$ such that the distance (error) between f and g is less than ε .

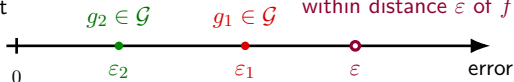


What denseness means:

For any $\varepsilon > 0$ (no matter how small), we can find a $g \in \mathcal{G}$ so that

$$d(f, g) < \varepsilon.$$

No matter how small ε is, some $g \in \mathcal{G}$ falls within distance ε of f .



In short: $\mathcal{G}$ is dense in $\mathcal{F} \iff \forall f \in \mathcal{F}, \forall \varepsilon > 0, \exists g \in \mathcal{G}$ such that $d(f, g) < \varepsilon$.

Continuous functions $C(\mathbb{R}^n)$:

$\mathcal{G}$ is a **universal approximator** $\iff$

$C(S) \simeq \mathcal{G}(S)$ for any compact $S \subset \mathbb{R}^n$

$\mathcal{G}_S$: restriction of functions in $\mathcal{G}$ to S .

SECTION 2

Elementary folklore construction

We will start with some **constructive results**, where we explicitly choose the form of the function used to approximate a given target function.

1. First discuss the univariate case where $f : [0, 1] \rightarrow \mathbb{R}$
2. Then discuss the extension to the multivariate case where $f : \mathbb{R}^d \rightarrow \mathbb{R}$

Theorem 1 ([Telgarsky(2021)])

Let $\epsilon > 0$ and assume $f : [0, 1] \rightarrow \mathbb{R}$ is ρ -Lipschitz and g is a 2-layer neural network with $\lceil \frac{\rho}{\epsilon} \rceil$ threshold nodes. Then we have

$$\sup_{x \in [0, 1]} |f(x) - g(x)| \leq \epsilon.$$

First present an auxiliary lemma that allows us to approximate continuous functions with **piecewise constant functions**.

Lemma 2

Let f, δ, ϵ be given as in Theorem 1. Assume we are given $U \subseteq \mathbb{R}^d$ and a partition $\mathcal{P}$ of U into rectangles of side lengths at most δ , i.e. $\mathcal{P} = (R_1, \dots, R_n)$. Then $\exists(\alpha_1, \dots, \alpha_n)$ such that $\sup_{\mathbf{x} \in U} |f(\mathbf{x}) - h(\mathbf{x})| \leq \epsilon$ where

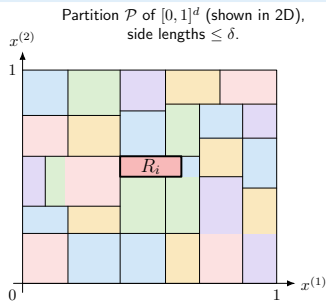
$$h(\mathbf{x}) = \sum_{i=1}^n \alpha_i \mathbf{1}_{R_i}(\mathbf{x}) \text{ and } \mathbf{1}_{R_i}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} \in R_i \\ 0 & \text{else.} \end{cases}$$

Theorem 3 ([Telgarsky(2021)])

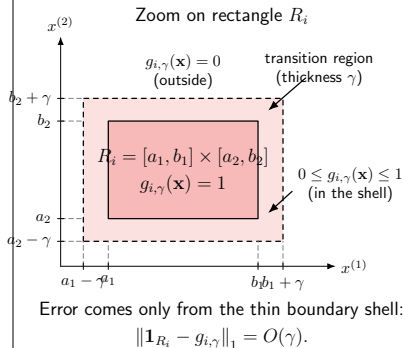
Consider a continuous function $f : \mathbb{R}^d \rightarrow \mathbb{R}$. Assume that, given $\epsilon > 0$, there exists $\delta > 0$ such that for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ with $\|\mathbf{x} - \mathbf{x}'\| \leq \delta$, then $|f(\mathbf{x}) - f(\mathbf{x}')| \leq \epsilon$. Then there exists a 3-layer neural network $g : \mathbb{R}^d \rightarrow \mathbb{R}$ with $\Omega(\delta^{-d})$ ReLU activation functions such that

$$\int_{[0,1]^d} |f(\mathbf{x}) - g(\mathbf{x})| dx \leq 2\epsilon.$$

(a) Partitioning the space



(b) Approximate one rectangle by a soft box



SECTION 3

Weierstrass Theorem

Theorem 4 (Weierstrass Theorem)

Polynomials $\mathcal{P}$ are dense in $C(I)$, where $I = [a; b]$ for any $a < b$.

Without loss of generality we can focus on $I = [0; 1]$. Define :

$$\phi : C([0; 1]) \rightarrow C([a; b]), \quad \phi(f) = f \circ \phi, \quad \phi = (1 - t)a + tb.$$

In plain words No matter what continuous function you pick on $[0, 1]$, you can always find some polynomial that is nearly identical to it

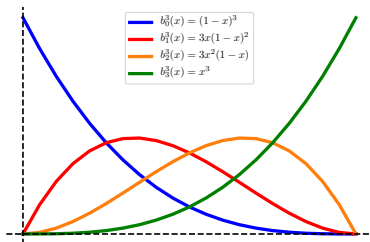
↪ difference can be made as small as you want, everywhere on that interval



Bernstein basis polynomials (degree m)

$$b_k^m(x) = \binom{m}{k} x^k (1-x)^{m-k}.$$

$$\Rightarrow \sum_{k=0}^m b_k^m(x) = 1 \quad \forall x \in [0; 1]$$



Define **approximation on lattice** with spacing $1/m$

$$q_m(x) = \sum_{k=0}^m f\left(\frac{k}{m}\right) b_k^m(x), \quad b_k^m(x) = \binom{m}{k} x^k (1-x)^{m-k}.$$

Consider residuals

$$|f(x) - q_m(x)| = \left| \sum_{k=0}^m r_k^m(x) \right|$$

$$r_k^m(x) := \left[f(x) - f\left(\frac{k}{m}\right) \right] b_k^m(x)$$

Upper bound by splitting

$$\mathcal{I} := \{k : |x - \frac{k}{m}| \leq \delta\}, \quad \text{and} \quad \mathcal{I}^c,$$

where δ is chosen such that $|f(x) - f(y)| \leq \epsilon/2$ and thus

$$\sum_{k \in \mathcal{I}} r_k^m(x) \leq \frac{\epsilon}{2} \sum_{k \in \mathcal{I}} b_k^m(x) \leq \frac{\epsilon}{2} \sum_{k=1}^m b_k^m(x) = \frac{\epsilon}{2}$$

Note that by concentration

$$\sum_{k \notin \mathcal{I}} r_k^m(x) \leq R \sum_{k \notin \mathcal{I}} b_k^m(x) \xrightarrow{m \rightarrow \infty} 0$$

and we can choose m to make it $< \epsilon/2$. (See lecture notes)

SECTION 4

Approximation with smooth functions

Let us now consider an **arbitrary smooth function** $\sigma \in C^\infty(\mathbb{R})$ and the span of its composition with affine functions

$$\begin{aligned}\mathcal{G}_\sigma^1 &= \{g : g(x) = \sigma(ax + b) \text{ for some } a, b \in \mathbb{R}\} \\ \mathcal{H}_\sigma^1 &= \text{span}(\mathcal{G}_\sigma^1)\end{aligned}$$

Then the following holds:

Theorem 5 (Leshno, Lin, Pinkus, Schocken 1993)

For any $\sigma \in C^\infty(\mathbb{R})$ not a polynomial. $\mathcal{H}_\sigma^1$ is a universal approximator.

1. Approximate derivative of σ : for all $h \neq 0$,

$$\frac{\sigma((a+h)x+b) - \sigma(ax+b)}{h} \in \mathcal{H}_\sigma^1$$

2. It follows that (generalizing to all k -th derivatives)

$$\left. \frac{d^k}{da^k} \sigma(ax+b) \right|_{a=0} = x^k \sigma^{(k)}(b) \in \text{cl}(\mathcal{H}_\sigma^1)$$

3. If there is always a b_0 such that $\sigma^{(k)}(b_0) \neq 0$ then we are guaranteed that $x^k \in \text{cl}(\mathcal{H}_\sigma^1)$ and hence all polynomials.
(known theorem)
4. By the **Weierstrass theorem** this implies the result.

Summary What we have shown now is that for the case of $n = 1$ (one-dimensional inputs), an MLP with smooth activation function σ is a universal approximator, unless σ is a polynomial.

This is because the linear output layer is picking an element in the span of the hidden units, each one of which computes a function in $\mathcal{G}_\sigma^1$.

Remark 1 (Role of activation function)

So as far as universal function approximation is concerned, there is nothing special about the logistic function or the hyperbolic tangent as choices of activation functions.

Define

$$\mathcal{G}_\sigma^n = \{g : g(\mathbf{x}) = \sigma(\boldsymbol{\theta} \cdot \mathbf{x}), \boldsymbol{\theta} \in \mathbb{R}^n\}, \quad \mathcal{H}_\sigma^n = \text{span}(\mathcal{G}_\sigma^n)$$

$$\mathcal{G}^n = \bigcup_{\sigma \in C(\mathbb{R})} \mathcal{G}_\sigma^n, \quad \mathcal{H}^n = \text{span}(\mathcal{G}^n)$$

Theorem 6 (Vostrecov and Kreines, 1961)

$\mathcal{H}^n$ is a universal function approximator.

3-layer neural networks with adaptive activation functions “could be” universal approximators. However, this in itself is not practical.
 $\leadsto$ We will soon see how to remove this limitation using the concept of **dimension lifting**.

Recall we showed earlier that for any $\sigma \in C^\infty(\mathbb{R})$ *not* a polynomial, $\mathcal{H}_\sigma^1$ is a universal approximator.

The following theorem allows us to extend the universality result we have for $n = 1$ to $\mathbb{R}^n$ for a single fixed $\sigma \in C^\infty(\mathbb{R})$.

Theorem 7 (Pinkus 1999)

For a **fixed** $\sigma \in C^\infty$,

$\mathcal{H}_\sigma^1$ universal for $C(\mathbb{R})$

$\implies \mathcal{H}_\sigma^n$ universal for $C(\mathbb{R}^n)$ for any $n \geq 1$

1. Fix f and compact $K \subset \mathbb{R}^n$. By Theorem 6, we can find ridge functions g_k s.t.

$$\left| f(\mathbf{x}) - \sum_{k=1}^m g_k(\boldsymbol{\theta}^k \cdot \mathbf{x}) \right| < \frac{\epsilon}{2} \quad (\forall \mathbf{x} \in K).$$

2. Since K is compact, $\boldsymbol{\theta}^k \cdot \mathbf{x} \in [\alpha_k, \beta_k]$ for $\mathbf{x} \in K$.
3. Because $\mathcal{H}_\sigma^1$ is dense in each $C([\alpha_k, \beta_k])$, Theorem 5 implies that we can find constants s.t.

$$\left| g_k(z) - \sum_{j=1}^{m_k} c_{kj} \sigma(a_{kj}z + b_{kj}) \right| \leq \frac{\epsilon}{2m} \quad (\forall k = 1, \dots, m)$$

4. Plugging things together yields the result.

- (1) For $n = 1$, an MLP with any continuous, non-polynomial activation function is a universal approximator.
- (2) Spans of ridge functions are universal approximators for $C(\mathbb{R}^n)$.
- (3) The non-linear part of any/each ridge function can be approximated according to (1).
- (4) Hence MLPs are universal function approximators.

SECTION 5

Complexity and Escaping the Curse of Dimensionality

Function classes represented by neural networks are rich (enough), but ...

- ▶ 1. How many units are required to obtain a desired approximation accuracy?
- ▶ 2. Is there an advantage of compositionality (multiple layers)?

For any absolutely integrable f , i.e. $f \in L^1$ ($\int_{\mathbb{R}^d} |f(\mathbf{x})| d\mathbf{x} < \infty$), define the Fourier transform of f as

$$\hat{f}(\omega) = \mathcal{F}f(\mathbf{x}) = \int_{\mathbb{R}^d} e^{-2\pi i \omega \cdot \mathbf{x}} f(\mathbf{x}) d\mathbf{x}, \quad \hat{f}: \mathbb{R}^d \rightarrow \mathbb{C}$$

- ▶ $\hat{f}(\omega)$ = result of the Fourier transform. It represents the function in the frequency domain: it tells us how much of each spatial frequency ω is present in $f(\cdot)$.
- ▶ $e^{-2\pi i \omega \cdot \mathbf{x}}$: This part essentially tests how well each spatial frequency component fits with the original function at each spatial position.

Regularity condition on Fourier transform $\widehat{g}$ of a function g

$$C_g := \int \|\omega\| |\widehat{g}(\omega)| d\omega < \infty$$

$C_g < \infty$: Fourier transformation of gradient function has to be **absolutely integrable**.

If g is differentiable, the Fourier transform of ∇g is given by

$$\Rightarrow \widehat{\nabla g}(\omega) = \omega \widehat{g}(\omega).$$

Condition on σ : bounded (measurable) and monotonic function σ such that $\sigma(t) \xrightarrow{t \rightarrow \infty} 1$ and $\sigma(t) \xrightarrow{t \rightarrow -\infty} 0$.

Theorem 8

For every $g : \mathbb{R}^d \rightarrow \mathbb{R}$ with **finite** C_g and any $r > 0$, there is a sequence of MLP functions $g_k(\mathbf{x})$ of the form

$$g_k(\mathbf{x}) = \sum_{j=1}^k \beta_j \sigma(\boldsymbol{\theta}_j \cdot \mathbf{x} + b_j) + b_0$$

such that

$$\int_{r\mathbb{B}} (g(\mathbf{x}) - g_k(\mathbf{x}))^2 \mu(d\mathbf{x}) \leq \mathcal{O}\left(\frac{1}{k}\right)$$

where $r\mathbb{B} = \{\mathbf{x} \in \mathbb{R}^d : \|\mathbf{x}\| \leq r\}$ and μ is any probability **measure** on $r\mathbb{B}$.

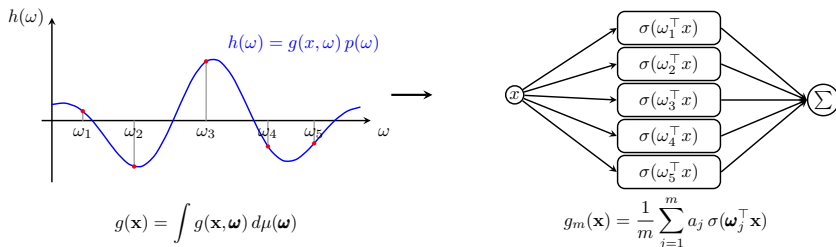
Main points of the theorem:

1. Lack of dependency on d . MLPs **do not suffer from the curse of dimensionality** when approximating (certain) functions.
2. Freedom in the choice of measure (data distribution)
3. Remarkable approximation error rate $\propto 1/m$.
4. Additional bounds and constraints on the parameters
5. Proof uses iterative construction: add units to fit residuals

Re: first point: linear combination of m basis functions has lower approximation error bound $(1/m)^{2/d}$ (much worse).

$\implies$ **Adaptivity of feature extraction is key!**

Barron's Theorem: proof idea



$$\text{independence} \implies \text{Var}\left(\frac{1}{m} \sum_{j=1}^m X_j\right) = \frac{\text{Var}(X_1)}{m}$$
$$\implies \mathbb{E}\left[\|g - g_m\|_{L^2(\mu)}^2\right] = \mathcal{O}\left(\frac{1}{m}\right)$$

Monte Carlo scaling:
squared error decreases like $1/m$

SECTION 6

Benefits of Depth

- ▶ Consistent empirical evidence: deeper network yield better approximations
- ▶ Classical results: focus on strength of shallow models (e.g. single hidden layer)
- ▶ **Do deep networks offer representational benefits?**
- ▶ No comprehensive theory exists of why deeper models are preferred and when.
- ▶ We will see some interesting pieces of the puzzle: e.g. **paradigmatic example of a function that is much easier to approximate with 2 hidden layers than 1.**

SECTION 6.1

Separation between Shallow and Deep Networks

Theorem 9 ([Telgarsky(2016)])

Let any integer $L \geq 1$ be given. There exists a ReLU neural network $f : \mathbb{R} \rightarrow \mathbb{R}$ with $3L^2 + 6$ nodes and $2L^2 + 4$ layers that can not be approximated by any ReLU network g with $\leq 2^L$ nodes and $\leq L$ layers such that

$$\int_{[0,1]} |f(x) - g(x)| \, dx \geq \frac{1}{32}.$$

In order to prove this theorem, we will...

- ▶ define a notion of complexity that depends on the number of oscillations in the function implemented by the neural network,
- ▶ show that this complexity measure grows polynomially in width, but exponentially in depth.

How do we measure oscillations in a function?

→ simply count the number of affine pieces

Formally, let $\mathcal{F}$ be the set of piecewise univariate linear mappings on R . Given a function $f \in \mathcal{F}$, we denote by $\delta_A(f)$ the number of affine pieces of f .

In order to study the number of oscillations in a neural network function, we will need the following properties of $\delta_A(f)$.

Proposition 10

1. For all $f \in \mathcal{F}$, and $\sigma(x) = \max(0, x)$, $\delta_A(\sigma(f)) \leq 2\delta_A(f)$,
2. For all $f_1, \dots, f_m \in \mathcal{F}$, $\delta_A(\sum_{i=1}^m f_i) \leq \sum_{i=1}^m \delta_A(f_i)$.

Any linear piece of f that does not cross the 0-axis either stays a linear piece (if above the 0-axis) or is mapped to zero. Only pieces that cross the 0-axis are separated into two pieces.

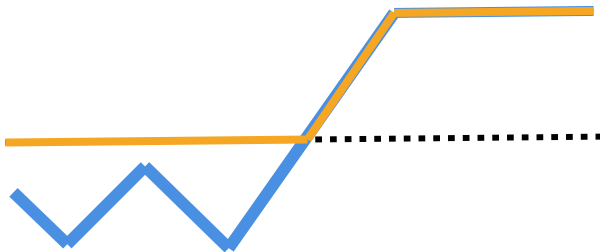
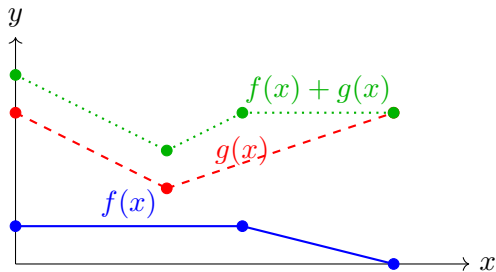


Figure: Illustration showing the effect of applying a ReLU function on a piecewise linear function. The dotted black line is the 0-axis. The piecewise linear function is shown in blue and the output of the ReLU function is shown in orange.

ii) When summing two piecewise functions $f(x) + g(x)$, there can only be a change in the slope at x if one of the functions f or g also had a change of slope at x .



Lemma 11

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a ReLU network with L layers of widths $(m_1, \dots, m_L)$ such that $m = \sum_{i=1}^L m_i$. Let $g : \mathbb{R} \rightarrow \mathbb{R}$ denote the output of some node in layer i as a function of the input. Then the number of affine pieces $\delta_A(g)$ satisfies

$$\delta_A(g) \leq 2^i \prod_{j < i} m_j.$$

The number of affine pieces in f satisfies $\delta_A(f) \leq \left(\frac{2m}{L}\right)^L$.

Proof.

See exercise session.



Main idea: We will create a highly oscillatory function f which we will approximate with a function g with few oscillations.

How? In order to create f , we will compose the following Δ function with itself such that the result of the composition increases its complexity (number of pieces).

$$\Delta(x) = 2\sigma_r(x) - 4\sigma_r(x - 1/2) + 2\sigma_r(x - 1) = \begin{cases} 2x & x \in [0, 1/2), \\ 2 - 2x & x \in [1/2, 1), \\ 0 & \text{otherwise,} \end{cases}$$

where

$$\sigma_r(x) = \max(0, x).$$

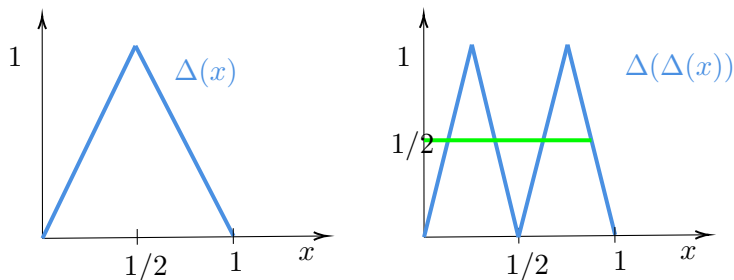


Figure: Illustration of the $\Delta(x)$ function as well as $\Delta(\Delta(x))$.

Try to compose the function with itself, $\Delta \circ \Delta$, what does the resulting function look like?

→ If you repeat this composition, you will see that Δ^L has 2^{L-1} copies of it self.

Consider the highly oscillatory blue function $f(x) = \Delta^{L^2+2}(x)$.

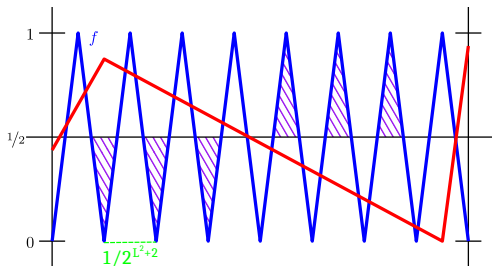


Figure: Source: [Telgarsky(2021)]

There are 2^{L^2+1} copies of $\Delta \implies 2^{L^2+2} - 1$ (half-)triangles since we get two triangles for each Δ but one lost on the boundary of $[0, 1]$.

Goal: approximate f with the red function $g \in \mathcal{F}$ which has few oscillations.

$$\int_{[0,1]} |f - g| \geq [\text{number surviving triangles}] \cdot [\text{area of triangle}]$$

- ▶ $g \in \mathcal{F}$ crosses the axis $x = \frac{1}{2}$ at most $\delta_A(g)$ times
- ▶ Number of half-triangles on one side of the $x \rightarrow \frac{1}{2}$ axis is larger than $2^{L^2+2} - 1 - 2\delta_A(g)$
- ▶ By Lemma 11 (with $m \leq 2^L$): $\delta_A(g) \leq (2 \cdot 2^L / L)^L \leq 2^{L^2}$
- ▶ Area of each triangle is $\frac{1}{4} \cdot \frac{1}{2^{L^2+2}} = 2^{-L^2-4}$

We obtain the following bound

$$\begin{aligned}\int_{[0,1]} |f - g| &\geq [\text{number surviving triangles}] \cdot [\text{area of triangle}] \\ &\geq \frac{1}{2} \left[2^{L^2+2} - 1 - 2 \cdot 2^{L^2} \right] \cdot \left[2^{-L^2-4} \right] \\ &= \frac{1}{2} \left[2^{L^2+1} - 1 \right] \cdot \left[2^{-L^2-4} \right] \\ &\geq \frac{1}{32}.\end{aligned}$$



Matus Telgarsky.

Benefits of depth in neural networks.

In **Conference on learning theory**, pages 1517–1539. PMLR, 2016.



Matus Telgarsky.

Deep learning theory lecture notes.

<https://mjt.cs.illinois.edu/dlt/>, 2021.

Version: 2021-10-27 v0.0-e7150f2d (alpha).